# K.R. Mangalam University

School of Engineering & Technology

Operating System Lab (ENCA252)

Lab Assignment 1

Implementation and Analysis of CPU Scheduling Algorithms (FCFS & SJF)


**Submitted by:**

Name: Anurag Kumar Mishra

Roll Number: 2401201076

Course: BCA (AI & DS)

**Submitted To:**

**Dr. Jyoti Yadav**

## 1. Objective

The primary goal of this lab exercise is to simulate and compare two fundamental CPU scheduling strategies — First Come First Serve (FCFS) and Shortest Job First (SJF). Both algorithms are evaluated using standard performance indicators: Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT), providing insights into their efficiency under varying workloads.

## 2. Tools & Technologies Used

| Tool | Purpose |
|---|---|
| Python 3.x | Core programming language for implementation |
| Built-in modules: copy, os, time | Standard utilities used in the project |
| Linux / macOS Terminal | Environment used to execute and test the program |
| GitHub | Source code hosting and version management |

## 3. Key Concepts

| Term | Formula | Description |
|---|---|---|
| CT | — | The moment at which a process completes its execution |
| TAT | CT − AT | Total elapsed time from process arrival to completion |
| WT | TAT − BT | Duration a process spends idle in the ready queue |

## 4. Task 1 — Process Creation & Input Handling

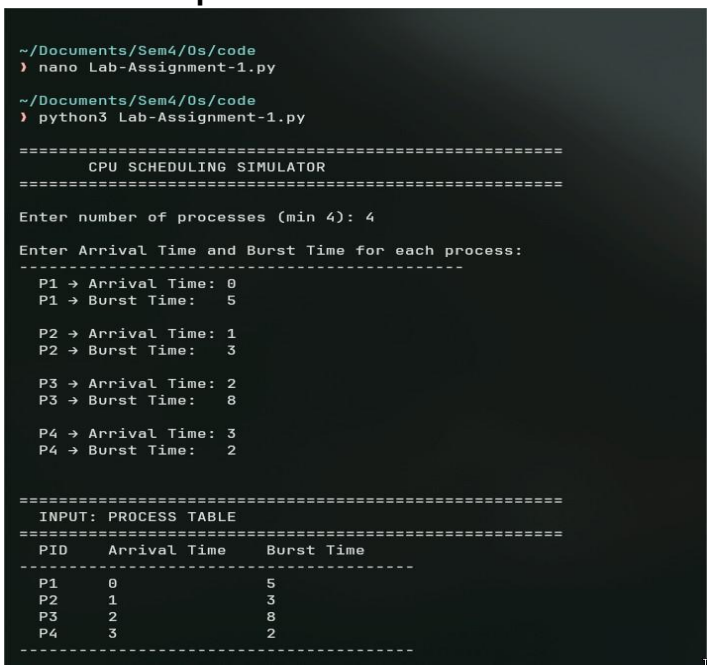
Every process in the simulation is represented using a Python class containing three core fields. User input is read interactively and stored as a list of process objects, which are then displayed in a structured tabular format before scheduling begins.

- **PID** — Unique identifier assigned to each process (e.g., P1, P2 ...)
- **AT** — Arrival Time: the point in time when the process enters the ready queue
- **BT** — Burst Time: total CPU time required by the process to complete

### Input Table

| PID | Arrival Time (AT) | Burst Time (BT) |
|---|---|---|
| P1 | 0 | 5 |
| P2 | 1 | 3 |
| P3 | 2 | 8 |
| P4 | 3 | 2 |

### Screenshot — Input Table



```
~/Documents/Sem4/Os/code
❯ nano Lab-Assignment-1.py

~/Documents/Sem4/Os/code
❯ python3 Lab-Assignment-1.py

=======================================================
       CPU SCHEDULING SIMULATOR
=======================================================

Enter number of processes (min 4): 4

Enter Arrival Time and Burst Time for each process:
-----------------------------------------------
  P1 → Arrival Time: 0
  P1 → Burst Time:   5

  P2 → Arrival Time: 1
  P2 → Burst Time:   3

  P3 → Arrival Time: 2
  P3 → Burst Time:   8

  P4 → Arrival Time: 3
  P4 → Burst Time:   2


=======================================================
  INPUT: PROCESS TABLE
=======================================================
  PID    Arrival Time    Burst Time
----------------------------------------
  P1     0               5
  P2     1               3
  P3     2               8
  P4     3               2
----------------------------------------
```

*Terminal output showing process input collection*

## 5. Task 2 — FCFS Scheduling

**First Come First Serve (FCFS)** is the most straightforward non-preemptive scheduling policy. Processes are dispatched to the CPU strictly in the sequence of their arrival. When no process is available, the CPU remains idle until the next one arrives.

### FCFS Results Table

| PID | AT | BT | CT | TAT | WT |
|---|---|---|---|---|---|
| P1 | 0 | 5 | 5 | 5 | 0 |
| P2 | 1 | 3 | 8 | 7 | 4 |
| P3 | 2 | 8 | 16 | 14 | 6 |
| P4 | 3 | 2 | 18 | 15 | 13 |

### Screenshot — FCFS Output

```
=====================================================
  FCFS — Results
=====================================================
  PID    AT     BT     CT     TAT    WT
---------------------------------------------
  P1     0      5      5      5      0
  P2     1      3      8      7      4
  P3     2      8      16     14     6
  P4     3      2      18     15     13
---------------------------------------------
```

*Terminal showing FCFS scheduling results*

## 6. Task 3 — SJF Scheduling (Non-Preemptive)

**Shortest Job First (SJF)** selects, from all currently available processes, the one with the smallest burst time. Being non-preemptive, once a process has been granted the CPU it runs uninterrupted until completion. Ties in burst time are resolved by comparing arrival times.

### SJF Results Table

| PID | AT | BT | CT | TAT | WT |
|---|---|---|---|---|---|
| P1 | 0 | 5 | 5 | 5 | 0 |
| P4 | 3 | 2 | 7 | 4 | 2 |
| P2 | 1 | 3 | 10 | 9 | 6 |
| P3 | 2 | 8 | 18 | 16 | 8 |

### Screenshot — SJF Output

```
=========================================================
  SJF - Results
=========================================================
  PID    AT     BT     CT     TAT    WT
---------------------------------------------
  P1     0      5      5      5      0
  P4     3      2      7      4      2
  P2     1      3      10     9      6
  P3     2      8      18     16     8
---------------------------------------------
```

*Terminal showing SJF scheduling results*

## 7. Task 4 — Gantt Chart Representation

### FCFS Gantt Chart

| P1 | P2 | P3 | P4 |
|---|---|---|---|

0 5 8 16 18

### SJF Gantt Chart

| P1 | P4 | P2 | P3 |
|---|---|---|---|

0 5 7 10 18

### Screenshot — Gantt Charts

```
============================================================
  TASK 4 - GANTT CHARTS
============================================================

  FCFS Gantt Chart
  --------------------------------
  |   P1   |   P2   |   P3   |   P4   |
  --------------------------------
  0       5       8       16      18

  SJF Gantt Chart
  --------------------------------
  |   P1   |   P4   |   P2   |   P3   |
  --------------------------------
  0       5       7       10      18
```

*Terminal output of both Gantt charts*

## 8. Task 5 — Performance Analysis & Comparison

| Metric | FCFS | SJF |
|---|---|---|
| Average Turnaround Time (TAT) | 10.25 | 8.50 |
| Average Waiting Time (WT) | 5.75 | 4.00 |

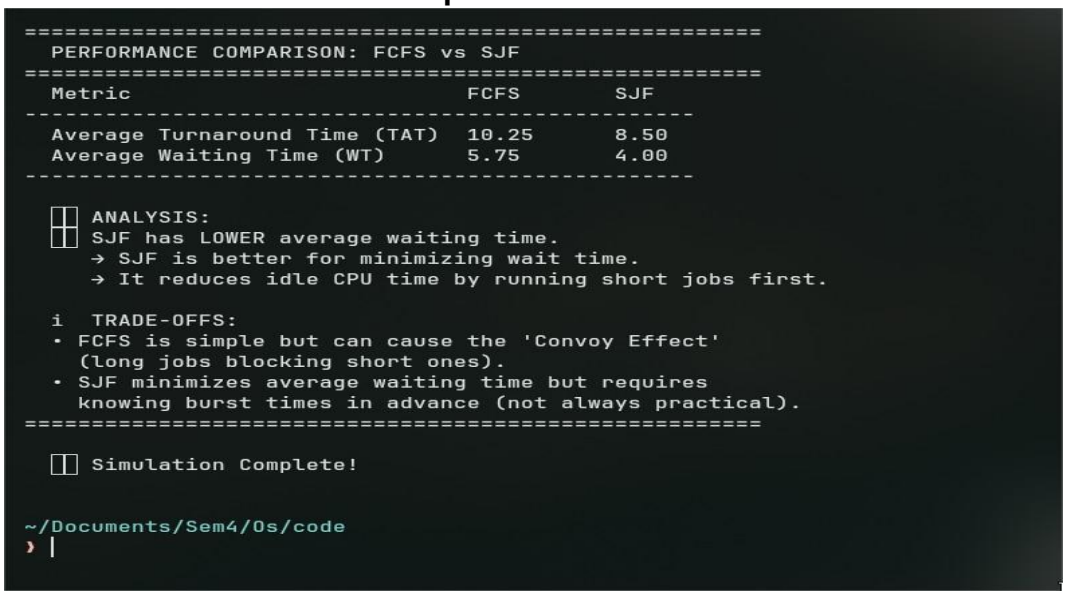
## Analysis

- **SJF** consistently yields a lower average waiting time when compared to FCFS.
- FCFS is susceptible to the **Convoy Effect**, where smaller processes are forced to wait behind longer ones that arrived earlier.
- SJF, while more efficient in reducing waiting time, depends on **advance knowledge of burst durations** — a requirement that is often impractical in real operating systems.

## Conclusion

**SJF outperforms FCFS** when the goal is to reduce average waiting time and turnaround time. Nonetheless, FCFS offers simplicity and fair ordering by arrival sequence. The optimal choice between the two ultimately depends on the nature of the workload and whether burst time information is available to the scheduler in advance.

## Screenshot — Performance Comparison



```
============================================================
  PERFORMANCE COMPARISON: FCFS vs SJF
============================================================
  Metric                          FCFS       SJF
--------------------------------------------------
  Average Turnaround Time (TAT)   10.25      8.50
  Average Waiting Time (WT)       5.75       4.00
--------------------------------------------------

  ANALYSIS:
  SJF has LOWER average waiting time.
    → SJF is better for minimizing wait time.
    → It reduces idle CPU time by running short jobs first.

  i  TRADE-OFFS:
  • FCFS is simple but can cause the 'Convoy Effect'
    (long jobs blocking short ones).
  • SJF minimizes average waiting time but requires
    knowing burst times in advance (not always practical).
============================================================

  Simulation Complete!

~/Documents/Sem4/Os/code
›
```

*Terminal showing performance metrics comparison*

## 9. GitHub Repository

Link:-[https://github.com/anuragkumarmishra2005/Operating-System-LabAssignment-1](https://github.com/anuragkumarmishra2005/Operating-System-LabAssignment-1)

## 10. Source Code

---

The complete source code is included below. It is also available in the linked GitHub repository under the filename *anurag_mishra_os_assignment1.py*.

```
# ================================================================
#  CPU Scheduling Simulator — FCFS, SJF & Round Robin
#  Course: ENCA252 | Program: BCA (AI & DS)
# ================================================================


# ================================================================
# SECTION 1: Job Class & Input Handling
# ================================================================


class Job:
    """
    Represents a single process/job in the CPU scheduler.

    Attributes:
        name       : Job identifier (e.g. J1, J2 ...)
        arrive     : Arrival time — when the job enters the ready queue
        duration   : Burst time  — how long the job needs on the CPU
        finish     : Completion time (set after scheduling)
        turnaround : TAT = finish - arrive
        wait       : Waiting time = turnaround - duration
    """
```

```
    def __init__(self, name, arrive, duration):
        self.name       = name
        self.arrive     = arrive
        self.duration   = duration
        self.finish     = 0
        self.turnaround = 0
        self.wait       = 0

def collect_jobs():
    """Prompt the user to enter job details; returns a list of Job objects."""

    print("\n" + "#" * 58)
    print("#        CPU SCHEDULING SIMULATOR v2.0                  #")
    print("#" * 58)

    while True:
        try:
            count = int(input("\nHow many jobs to schedule? (minimum 4): "))
            if count < 4:
                print("  [!] Please enter at least 4 jobs.")
            else:
                break
        except ValueError:
            print("  [!] That's not a valid number. Try again.")

    jobs = []
    print(f"\nEnter Arrival Time & Burst Time for each job:")
    print("-" * 50)

    for idx in range(1, count + 1):
        while True:
            try:
                arrive   = int(input(f"  J{idx} -> Arrival Time : "))
                duration = int(input(f"  J{idx} -> Burst Time   : "))
```

```
                if arrive < 0 or duration <= 0:
                    print("  [!] Arrival >= 0 and Burst Time must be > 0.")
                    continue
                jobs.append(Job(f"J{idx}", arrive, duration))
                print()
                break
            except ValueError:
                print("  [!] Whole numbers only, please.")

    return jobs

def display_table(jobs, heading="Results"):
    """Print scheduling results in a formatted table."""

    col = "-" * 52
    print(f"\n  [{heading}]")
    print(col)
    print(f"  {'Job':<6} {'Arrive':<8} {'Burst':<8} {'Finish':<8} {'TAT':<8} {'Wait':<8}")
    print(col)
    for j in jobs:
        print(f"  {j.name:<6} {j.arrive:<8} {j.duration:<8} {j.finish:<8} {j.turnaround:<8} {j.wait:<8}")
    print(col)

# ================================================================
# SECTION 2: FCFS — First Come First Served
# ================================================================

def run_fcfs(jobs):
    """
    First Come First Served (Non-Preemptive):
    - Sort all jobs by arrival time.
    - Run each in order; handle CPU idle gaps.
    - Compute Finish, TAT, and Wait for every job.
```

```
    Returns: (ordered list of jobs, gantt timeline)
    """
    ordered = sorted(jobs, key=lambda j: j.arrive)
    clock   = 0
    timeline = []   # Each entry: (label, start, end)

    for j in ordered:
        if clock < j.arrive:
            timeline.append(("---", clock, j.arrive))   # CPU idle
            clock = j.arrive

        start  = clock
        clock += j.duration

        j.finish     = clock
        j.turnaround = j.finish - j.arrive
        j.wait       = j.turnaround - j.duration

        timeline.append((j.name, start, j.finish))

    return ordered, timeline

# ================================================================
# SECTION 3: SJF — Shortest Job First (Non-Preemptive)
# ================================================================

def run_sjf(jobs):
    """
    Shortest Job First (Non-Preemptive):
    - At each scheduling point, look at all arrived jobs.
    - Pick the one with the smallest burst time.
    - Ties broken by earliest arrival (FCFS within same burst).
```

```
Returns: (completed jobs in execution order, gantt timeline)
"""
import copy

pool      = copy.deepcopy(jobs)   # Work on copies to preserve originals
clock     = 0
done      = []
timeline  = []

while pool:
    arrived = [j for j in pool if j.arrive <= clock]

    if not arrived:
        # Jump to the next job's arrival — CPU is idle
        next_time = min(pool, key=lambda j: j.arrive).arrive
        timeline.append(("---", clock, next_time))
        clock = next_time
        continue

    # Pick job with shortest burst; tie-break by arrival time
    chosen = min(arrived, key=lambda j: (j.duration, j.arrive))

    start  = clock
    clock += chosen.duration

    chosen.finish     = clock
    chosen.turnaround = chosen.finish - chosen.arrive
    chosen.wait       = chosen.turnaround - chosen.duration

    timeline.append((chosen.name, start, chosen.finish))
    done.append(chosen)
    pool.remove(chosen)

# Write results back to the original job objects
```

```
    for orig in jobs:
        for finished in done:
            if orig.name == finished.name:
                orig.finish     = finished.finish
                orig.turnaround = finished.turnaround
                orig.wait       = finished.wait

    return done, timeline

# ================================================================
# SECTION 4: Round Robin Scheduling (Bonus)
# ================================================================

def run_round_robin(jobs, quantum=2):
    """
    Round Robin (Preemptive, time-sliced):
    - Each job gets at most 'quantum' units of CPU time per turn.
    - If not finished, it re-enters the back of the ready queue.
    - Fair scheduling — no job starves.

    Parameter:
        quantum : Time slice size (default = 2 units)

    Returns: (jobs with updated metrics, gantt timeline)
    """
    import copy

    pool     = copy.deepcopy(jobs)
    for j in pool:
        j.remaining = j.duration   # Track how much CPU time is still needed

    clock    = 0
    queue    = []
    timeline = []
```

```
visited  = set()

# Start with jobs that arrive at time 0
queue = [j for j in sorted(pool, key=lambda j: j.arrive) if j.arrive <= clock]
remaining_pool = [j for j in pool if j.arrive > clock]

while queue or remaining_pool:
    if not queue:
        # Nothing in the queue — jump to next arrival
        next_job = min(remaining_pool, key=lambda j: j.arrive)
        timeline.append(("---", clock, next_job.arrive))
        clock = next_job.arrive
        # Add all jobs that have now arrived
        newly_arrived = [j for j in remaining_pool if j.arrive <= clock]
        queue.extend(newly_arrived)
        for j in newly_arrived:
            remaining_pool.remove(j)

    current = queue.pop(0)
    run_for = min(quantum, current.remaining)
    start   = clock
    clock  += run_for
    current.remaining -= run_for

    timeline.append((current.name, start, clock))

    # Add any new arrivals during this slice
    newly_arrived = [j for j in remaining_pool if j.arrive <= clock]
    for j in newly_arrived:
        queue.append(j)
        remaining_pool.remove(j)

    if current.remaining == 0:
        current.finish     = clock
```

```
            current.turnaround = current.finish - current.arrive
            current.wait       = current.turnaround - current.duration
        else:
            queue.append(current)   # Re-queue unfinished job

    # Copy RR results back to original jobs
    for orig in jobs:
        for p in pool:
            if orig.name == p.name:
                orig.finish     = p.finish
                orig.turnaround = p.turnaround
                orig.wait       = p.wait

    return pool, timeline

# ================================================================
# SECTION 5: Gantt Chart Printer
# ================================================================

def draw_gantt(timeline, heading="Gantt Chart"):
    """
    Draws a text-based Gantt chart.

    Example:
    | J1  | ---  | J3  | J2  |
      0     3      5     7    11
    """
    print(f"\n  >> {heading}")
    bar = "  +" + "+".join("-" * 8 for _ in timeline) + "+"
    print(bar)

    label_row = "  |"
    for (name, start, end) in timeline:
        label_row += f" {name:^6} |"
```

```
    print(label_row)

    print(bar)

    time_row = "  "
    for (name, start, end) in timeline:
        time_row += str(start).ljust(9)
    time_row += str(timeline[-1][2])
    print(time_row)

# ================================================================
# SECTION 6: Performance Comparison
# ================================================================

def compare_algorithms(fcfs_jobs, sjf_jobs, rr_jobs):
    """
    Calculates and prints average TAT and WT for all three algorithms,
    then gives a plain-English analysis of the results.
    """
    n = len(fcfs_jobs)

    def averages(job_list):
        avg_tat = sum(j.turnaround for j in job_list) / n
        avg_wt  = sum(j.wait       for j in job_list) / n
        return round(avg_tat, 2), round(avg_wt, 2)

    fcfs_tat, fcfs_wt = averages(fcfs_jobs)
    sjf_tat,  sjf_wt  = averages(sjf_jobs)
    rr_tat,   rr_wt   = averages(rr_jobs)

    print("\n" + "#" * 58)
    print("#         PERFORMANCE COMPARISON REPORT                #")
    print("#" * 58)
    print(f"\n  {'Metric':<32} {'FCFS':>7} {'SJF':>7} {'RR(q=2)':>9}")
```

```
    print("  " + "-" * 55)
    print(f"  {'Avg Turnaround Time (TAT)':<32} {fcfs_tat:>7} {sjf_tat:>7} {rr_tat:>9}")
    print(f"  {'Avg Waiting Time (WT)':<32} {fcfs_wt:>7} {sjf_wt:>7} {rr_wt:>9}")
    print("  " + "-" * 55)

    # Find best algorithm for each metric
    best_tat = min(fcfs_tat, sjf_tat, rr_tat)
    best_wt  = min(fcfs_wt,  sjf_wt,  rr_wt)

    tat_names = {fcfs_tat: "FCFS", sjf_tat: "SJF", rr_tat: "Round Robin"}
    wt_names  = {fcfs_wt:  "FCFS", sjf_wt:  "SJF", rr_wt:  "Round Robin"}

    print(f"\n  BEST avg TAT  -->  {tat_names[best_tat]}")
    print(f"  BEST avg WT   -->  {wt_names[best_wt]}")

    print("\n  ANALYSIS:")
    print("  * FCFS  : Simple & fair in order of arrival, but long jobs")
    print("            can block shorter ones (Convoy Effect).")
    print("  * SJF   : Best average waiting time overall, but needs")
    print("            burst time known in advance (impractical in real OS).")
    print("  * RR    : Best for fairness & time-sharing systems.")
    print("            Quantum size directly affects performance —")
    print("            too small = high overhead; too large = behaves like FCFS.")
    print("#" * 58)

# ================================================================
# MAIN
# ================================================================

def main():
    import copy

    # --- Input ---
```

```
jobs = collect_jobs()

print("\n" + "#" * 58)
print("#              ENTERED JOB TABLE                       #")
print("#" * 58)
print(f"  {'Job':<6} {'Arrival Time':<16} {'Burst Time':<12}")
print("  " + "-" * 38)
for j in jobs:
    print(f"  {j.name:<6} {j.arrive:<16} {j.duration:<12}")
print("  " + "-" * 38)

# Make independent copies for each algorithm
fcfs_input = copy.deepcopy(jobs)
sjf_input  = copy.deepcopy(jobs)
rr_input   = copy.deepcopy(jobs)

# --- FCFS ---
fcfs_result, fcfs_timeline = run_fcfs(fcfs_input)
display_table(fcfs_result, heading="FCFS — Scheduling Results")
draw_gantt(fcfs_timeline, heading="FCFS Gantt Chart")

# --- SJF ---
sjf_result, sjf_timeline = run_sjf(sjf_input)
display_table(sjf_result, heading="SJF — Scheduling Results")
draw_gantt(sjf_timeline, heading="SJF Gantt Chart")

# --- Round Robin ---
quantum = 2
try:
    q_input = input("\nEnter Time Quantum for Round Robin (default 2): ").strip()
    if q_input:
        quantum = int(q_input)
        if quantum <= 0:
            raise ValueError
```

```
    except ValueError:
        print("  [!] Invalid quantum. Using default value of 2.")
        quantum = 2

    rr_result, rr_timeline = run_round_robin(rr_input, quantum=quantum)
    display_table(rr_result, heading=f"Round Robin (q={quantum}) — Scheduling
Results")
    draw_gantt(rr_timeline,  heading=f"Round Robin (q={quantum}) Gantt Chart")

    # --- Comparison ---
    compare_algorithms(fcfs_result, sjf_result, rr_result)

    print("\n  Simulation complete!\n")

if __name__ == "__main__":
    main()
```